SOFTWARE PROJECT • PADRÕES DE PROJETO (GoF)

Padrões de Projeto Estruturais

Como combinar classes e objetos em estruturas maiores, mantendo-as flexíveis e eficientes

Adapter | Bridge | Composite | Decorator | Facade | Flyweight | Proxy

Prof. Baldoino Fonseca • Universidade Federal de Alagoas • Programação Orientada a Objetos e Padrões de Projeto

ROTEIRO

O que vamos ver nesta aula

1 Fundamentos

- Dos criacionais aos estruturais
- Herança × composição de objetos
- Panorama dos sete padrões

2 Os sete padrões

- Adapter, Bridge, Composite
- Decorator, Facade, Flyweight, Proxy
- Cada um: motivação, problema, intenção, UML, código Java e quando usar

3 Síntese e prática

- Onde aparecem no JDK e em frameworks
- Sinais de alerta no código
- Como distinguir os “embrulhos” e atividade em sala

Mesma estrutura de cada padrão dos criacionais: motivação → problema → intenção → estrutura (UML) → código → quando usar.

CONTEXTO

Onde estamos: as três famílias de padrões GoF

Criacionais

AULA ANTERIOR ✓

Como criar objetos sem acoplar o código ao new?

Factory Method • Abstract Factory • Builder • Prototype • Singleton

Estruturais

ESTA AULA

Como combinar classes e objetos em estruturas maiores sem perder flexibilidade?

Adapter • Bridge • Composite • Decorator • Facade • Flyweight • Proxy

Comportamentais

PRÓXIMA AULA

Como os objetos se comunicam e dividem responsabilidades?

Strategy • Observer • Command • State • Template Method • …

Os padrões estruturais tratam de como classes e objetos se compõem para formar estruturas maiores (GAMMA et al., 1994).

FUNDAMENTO

Herança × composição: duas formas de reutilizar

Exemplo: como um Carro pode ter um motor elétrico em vez de um motor a combustão?

Herança

relação "é um"

CarroEletrico.java

```
class Carro {
    void acelerar() { /* motor a combustão */ }
}
class CarroEletrico extends Carro {
    @Override
    void acelerar() { /* motor elétrico */ }
}

// novo tipo de motor = nova subclasse de Carro
```

- **Decidida ao escrever o código:** um CarroEletrico será elétrico para sempre.
- **Acoplamento forte:** o filho depende dos detalhes do pai.
- **Variações viram subclasses:** motor × câmbio × tração = explosão de classes.

Composição de objetos

relação "tem um"

Motor.java • Carro.java

```
interface Motor { void acelerar(); }

class MotorEletrico implements Motor {
    public void acelerar() { /* motor elétrico */ }
}
class MotorCombustao implements Motor { /* ... */ }

class Carro {
    private final Motor motor;      // "tem um" Motor
    Carro(Motor m) { this.motor = m; }
    void acelerar() { motor.acelerar(); }   // delega
}
new Carro(new MotorEletrico());     // escolhido em execução
```

- **Montada em execução:** o mesmo Carro aceita qualquer Motor.
- **Acoplamento fraco:** depende só da interface Motor.
- **Variações viram objetos** que se combinam sem criar novas classes.

Regra prática: herança para um "é um" verdadeiro e estável; composição quando o comportamento precisa variar ou ser combinado. Quase todos os padrões estruturais usam composição.

"Dê preferência à composição de objetos em vez da herança de classes." (GAMMA et al., 1994; BLOCH, 2018, Item 18)

PANORAMA

Os sete padrões estruturais em uma pergunta

Adapter

traduz

“Essa interface não combina com a que eu preciso.”

Bridge

separa

“Duas dimensões variam de forma independente.”

Composite

agrega

“Parte e todo devem ser tratados do mesmo jeito.”

Decorator

embrulha

“Quero acrescentar comportamento em tempo de execução.”

Facade

simplifica

“O subsistema é complicado demais para os clientes.”

Flyweight

compartilha

“Objetos demais, memória de menos.”

Proxy

representa

“Preciso controlar o acesso a este objeto.”

Atenção

Adapter, Decorator, Facade e Proxy “embrulham” outro objeto. A estrutura é parecida; o que muda é a intenção.

PADRÃO ESTRUTURAL 1 DE 7

Adapter

Converter a interface de uma classe na interface que o cliente espera

01

AD BR CP DC FC FW PX

Motivação: pagamentos na loja online



Pagamentos da loja online

Sua loja online já tem um checkout estável: o carrinho chama pagar(valor, cartao) e o pedido é cobrado.

Para vender no exterior, a empresa contrata uma operadora internacional. Mas o SDK dela fala outro “idioma”: cobra em centavos, exige código de moeda e devolve um número como status.

Você não pode editar o SDK (é de terceiros) e não quer reescrever o checkout inteiro. Falta um tradutor entre os dois lados.

> *Como ligar o checkout ao SDK da operadora sem alterar nenhum dos dois? Vamos ver o que acontece quando o checkout “aprende” o idioma do SDK.*

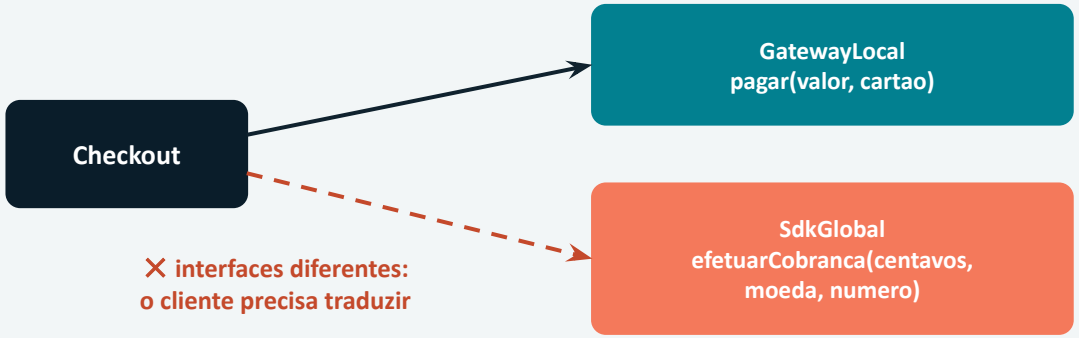
O problema: o checkout fala o idioma do SDK

```
void finalizar(Pedido p, String operadora) {
    if (operadora.equals("local")) {
        gatewayLocal.pagar(p.total(), p.cartao());
    } else {
        long centavos = Math.round(p.total() * 100);
        int st = sdkGlobal.efetuarCobranca(centavos, "BRL", p.cartao().numero());
        if (st != 0) throw new PagamentoException(st);
    }
}
```

✕ Sem o padrão

- Checkout conhece detalhes do SDK: centavos, moeda, código de status
- Cada nova operadora acrescenta mais um else-if
- Regras de conversão duplicadas em todo ponto que cobra
- Impossível testar o checkout sem o SDK real

O QUE ESTÁ ACONTECENDO



Intenção: traduzir uma interface em outra

> *"Converter a interface de uma classe em outra interface esperada pelos clientes. O Adapter permite que classes trabalhem juntas, o que não seria possível por causa de interfaces incompatíveis."*

GAMMA, HELM, JOHNSON, VLISSIDES (1994) — Intent (tradução livre)

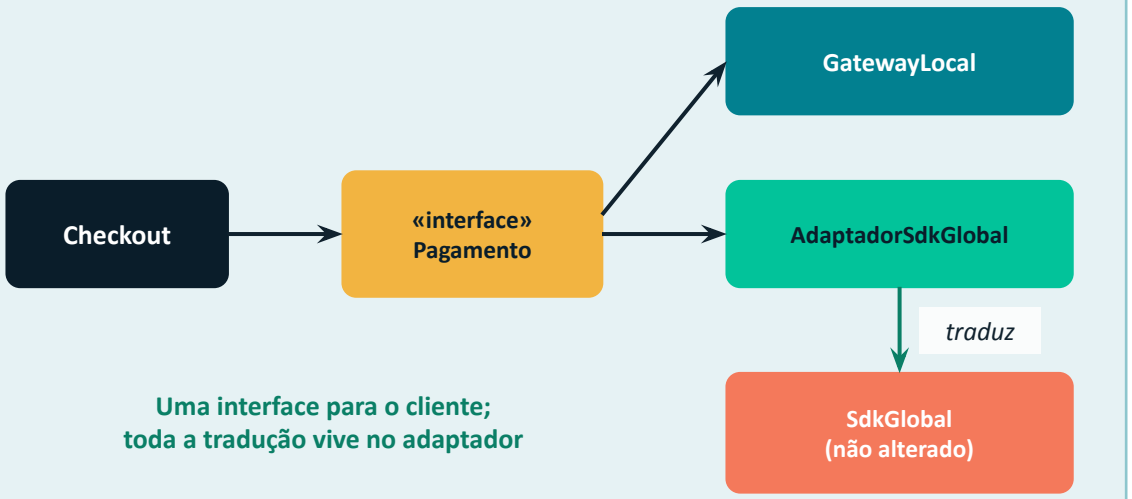
O cliente continua falando uma única interface (o Target). O Adapter implementa essa interface e, por dentro, converte cada chamada para o formato do objeto adaptado (Adaptee).

Nem o cliente nem o SDK mudam: a tradução fica concentrada em uma classe.

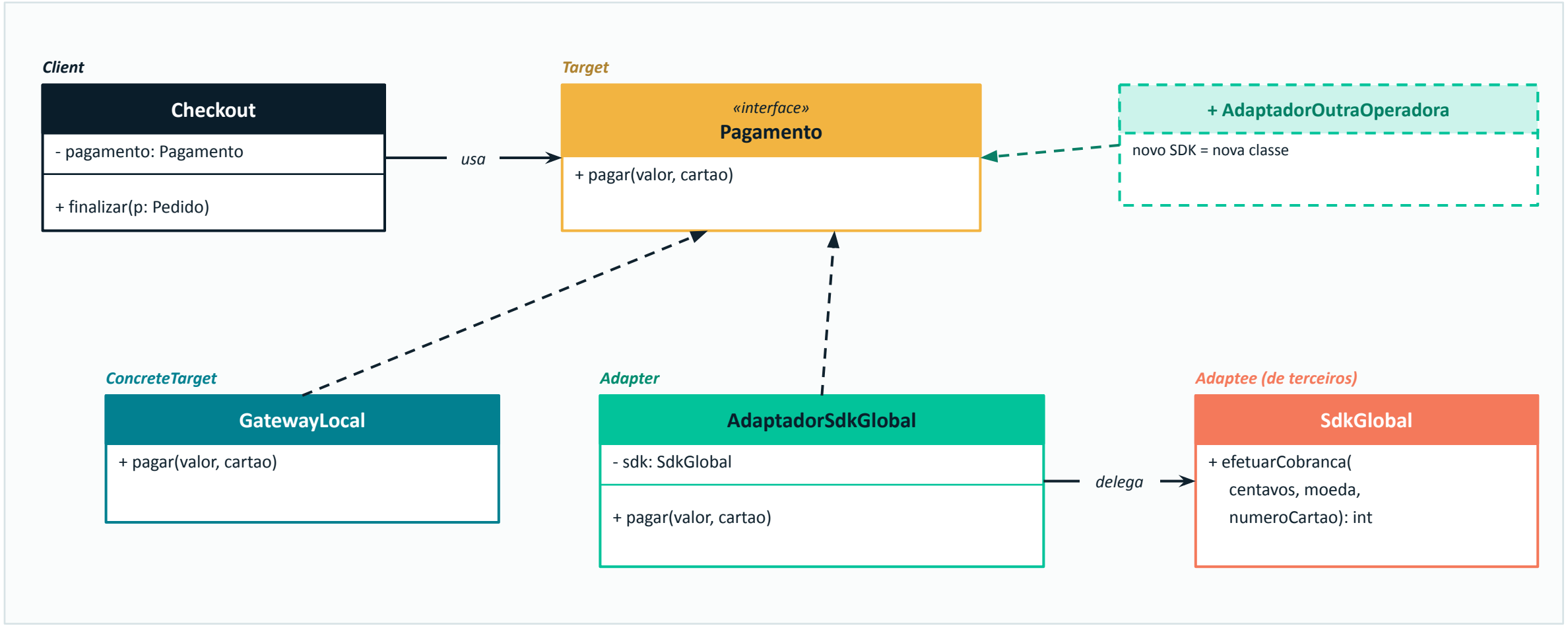
DUAS FORMAS DE IMPLEMENTAR

- **Adapter de objeto (composição):** guarda uma referência ao Adaptee — a forma preferida em Java.
- **Adapter de classe (herança):** estende o Adaptee; exige herança múltipla, limitada em Java.

A SOLUÇÃO EM AÇÃO



Estrutura: o Adapter implementa o Target e delega ao Adaptee



Cada SDK incompatível ganha 1 adaptador; o Checkout continua dependendo apenas de Pagamento.

Código: o Checkout usa Pagamento; o adaptador traduz para o SDK

Pagamento.java • Checkout.java • SdkGlobal.java

```java
// Target — o que o cliente espera
public interface Pagamento {
    void pagar(double valor, Cartao cartao);
}

// Client — depende só do Target
public class Checkout {
    private final Pagamento pagamento;
    public Checkout(Pagamento p) { this.pagamento = p; }
    public void finalizar(Pedido p) {
        pagamento.pagar(p.total(), p.cartao());
    }
}

// Adaptee — SDK de terceiros (não podemos alterar)
public class SdkGlobal {
    public int efetuarCobranca(long centavos,
            String moeda, String numeroCartao) {
        return 0;   // 0 = aprovado (corpo omitido)
```

AdaptadorSdkGlobal.java • Montagem

```java
// Adapter — implementa Target, usa Adaptee
public class AdaptadorSdkGlobal implements Pagamento {
    private final SdkGlobal sdk;
    public AdaptadorSdkGlobal(SdkGlobal sdk) {
        this.sdk = sdk;
    }

    @Override
    public void pagar(double valor, Cartao cartao) {
        long centavos = Math.round(valor * 100);
        int status = sdk.efetuarCobranca(
                centavos, "BRL", cartao.numero());
        if (status != 0)
            throw new PagamentoException(status);
    }}
Pagamento pg = new AdaptadorSdkGlobal(new SdkGlobal());
Checkout checkout = new Checkout(pg);
checkout.finalizar(pedido);    // → pagar() → efetuarCobranca()
```

Fluxo: Checkout ⟶ Pagamento.pagar() ⟶ AdaptadorSdkGlobal (converte) ⟶ SdkGlobal.efetuarCobranca(). Outra operadora = outro Pagamento injetado.

Quando usar (e quando evitar) o Adapter

✓ Use quando

- Você quer usar uma classe existente, mas a interface dela não combina com a que o código espera
- Reutilizar bibliotecas legadas ou de terceiros que não podem ser modificadas
- Isolar o sistema de mudanças em um SDK externo (camada anticorrupção)
- Integrar várias fontes parecidas atrás de uma única interface

! Cuidado / evite quando

- Se você controla as duas interfaces, ajuste uma delas: o adapter é uma camada a mais
- Muitos adapters pequenos podem esconder uma interface mal definida
- Adaptar semânticas muito diferentes (assíncrono → síncrono, por exemplo) vaza detalhes e esconde custos
- Conversões com perda de informação precisam ser documentadas e testadas

PADRÕES RELACIONADOS

Bridge (separa abstração e implementação de propósito, desde o início) • Decorator (mesma interface, adiciona comportamento) • Proxy (mesma interface, controla acesso) • Facade (cria uma interface nova e mais simples para um subsistema)

PADRÃO ESTRUTURAL 2 DE 7

Bridge

02

Desacoplar uma abstração de sua implementação para que as duas possam variar independentemente

AD | BR | CP | DC | FC | FW | PX

Motivação: a central de avisos da universidade



Central de avisos

O sistema acadêmico precisa avisar os alunos. Existem três tipos de mensagem: Alerta (urgente), Lembrete (agendado) e Promoção (cantina, eventos).

Cada mensagem pode ser entregue por três canais: e-mail, SMS ou push. E amanhã alguém vai pedir WhatsApp.

Duas coisas variam por motivos diferentes: o tipo da mensagem muda por regra de negócio; o canal muda por tecnologia.

> *Tipo de mensagem e canal de entrega são dois eixos independentes de variação. O que acontece se modelarmos os dois com herança?*

O problema: uma subclasse para cada combinação

```
abstract class Notificacao { abstract void enviar(String msg); }
class AlertaEmail    extends Notificacao { void enviar(String m) { /* "[URGENTE] " + SMTP */ } }
class AlertaSms      extends Notificacao { void enviar(String m) { /* "[URGENTE] " + gateway SMS */ } }
class AlertaPush     extends Notificacao { void enviar(String m) { /* "[URGENTE] " + push */ } }
class LembreteEmail extends Notificacao { void enviar(String m) { /* "[LEMBRETE] " + SMTP */ } }
class LembreteSms    extends Notificacao { void enviar(String m) { /* "[LEMBRETE] " + gateway SMS */ } }
// ... LembretePush, PromocaoEmail, PromocaoSms, PromocaoPush  →  3 tipos × 3 canais = 9 classes
// Novo canal (WhatsApp)? +3 classes.   Novo tipo (Comunicado)? +3 classes.
```

✕ Sem o padrão

- 3 tipos × 3 canais = 9 classes (e crescendo)
- Novo canal ou novo tipo multiplica as classes
- Código de envio de cada canal copiado em cada tipo
- Corrigir o SMTP exige editar várias classes

O QUE ESTÁ ACONTECENDO



9 classes hoje → 12 com WhatsApp → 15 com um novo tipo

Intenção: duas hierarquias ligadas por uma ponte

> *"Desacoplar uma abstração de sua implementação, de modo que as duas possam variar independentemente."*

GAMMA, HELM, JOHNSON, VLISSIDES (1994) — Intent (tradução livre)

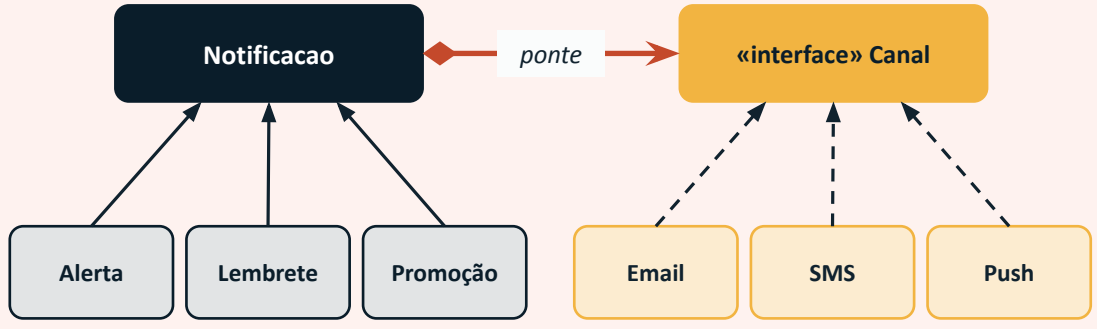
Em vez de uma hierarquia com todas as combinações, criamos duas hierarquias: a abstração (tipo de mensagem) e a implementação (canal).

A ponte é uma referência: a abstração guarda um Canal e delega a ele o trabalho de entregar.

O QUE MUDA NA PRÁTICA

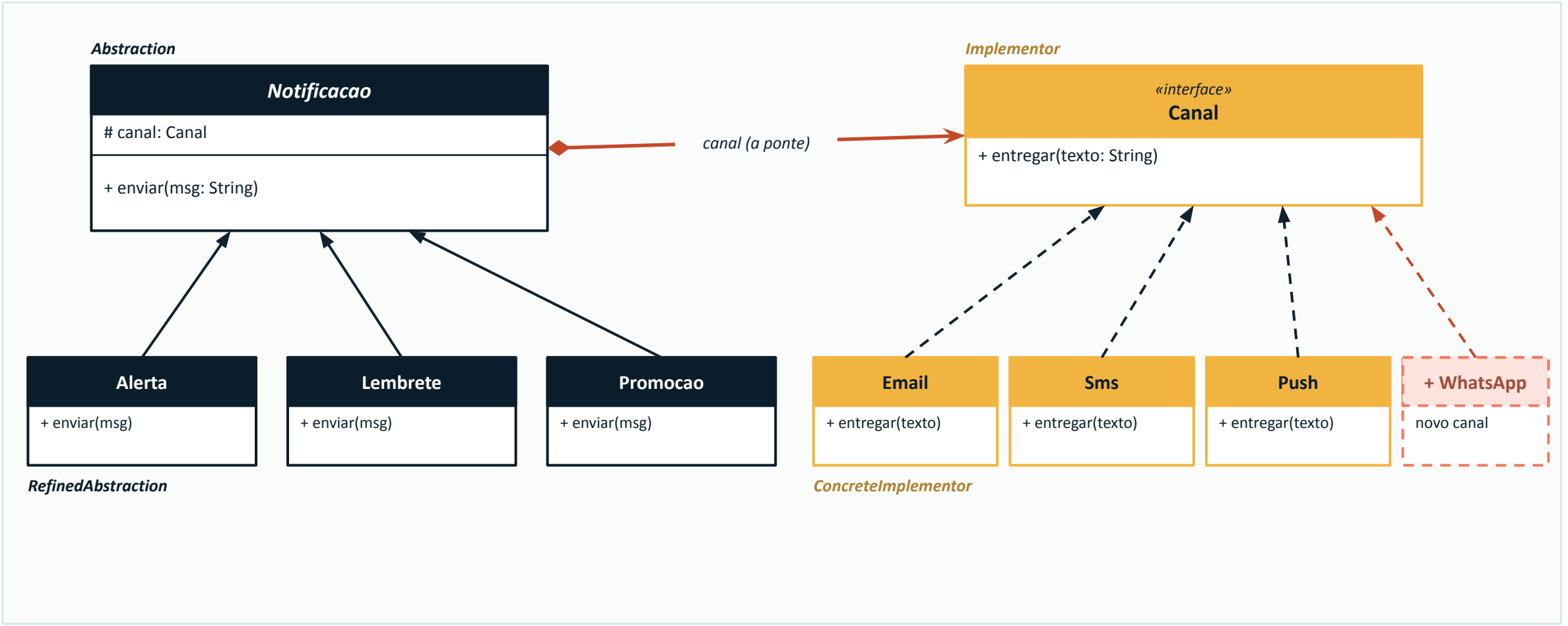
- **Novo canal:** 1 classe nova em Canal; nenhum tipo é alterado.
- **Novo tipo:** 1 classe nova em Notificacao; nenhum canal é alterado.
- **Combinação:** montada em tempo de execução, no construtor.

A SOLUÇÃO EM AÇÃO



3 + 3 = 6 classes • WhatsApp = +1 classe

Estrutura: Notificacao delega a um Canal pela ponte



Novo tipo = 1 subclasse de Notificacao; novo canal = 1 implementação de Canal. As combinações não viram classes.

Código: Notificacao guarda um Canal e delega a entrega

Canal.java • Email.java • Sms.java

```java
// Implementor — dimensão "canal de entrega"
public interface Canal {
    void entregar(String texto);
}

// ConcreteImplementors
public class Email implements Canal {
    public void entregar(String texto) {
        /* envia via SMTP */
    }
}
public class Sms implements Canal {
    public void entregar(String texto) {
        /* envia via gateway de SMS */
    }
}
// Push, WhatsApp... seguem o mesmo molde
```

Notificacao.java • Alerta.java • Cliente

```java
// Abstraction — dimensão "tipo de mensagem"
public abstract class Notificacao {
    protected final Canal canal;          // a ponte
    protected Notificacao(Canal canal) { this.canal = canal; }
    public abstract void enviar(String msg);
}

// RefinedAbstraction
public class Alerta extends Notificacao {
    public Alerta(Canal c) { super(c); }
    public void enviar(String msg) {
        canal.entregar("[URGENTE] " + msg);
    }
}
new Alerta(new Sms()).enviar("Prova amanhã");
new Lembrete(new Email()).enviar("Matrícula abre");
```

WhatsApp = 1 classe nova em Canal; Comunicado = 1 classe nova em Notificacao. Nenhuma classe existente é alterada.

Quando usar (e quando evitar) o Bridge

✓ Use quando

- Há duas dimensões independentes de variação (tipo × canal, forma × cor, interface gráfica × plataforma)
- Você quer evitar a explosão combinatória de subclasses
- A implementação precisa ser trocada em tempo de execução
- Várias abstrações compartilham a mesma implementação

! Cuidado / evite quando

- Existe só uma dimensão de variação: herança simples basta
- É preciso antecipar os dois eixos no design; separar depois é uma refatoração custosa
- Mais indireção: duas hierarquias para entender, testar e manter
- A fronteira entre abstração e implementação pode ser artificial em domínios pequenos

PADRÕES RELACIONADOS

Adapter (resolve incompatibilidade depois que as classes existem; o Bridge é planejado antes) • Abstract Factory (cria o par abstração + implementação correto) • Strategy (mesma estrutura, mas o foco é trocar um algoritmo) • State

PADRÃO ESTRUTURAL 3 DE 7

Composite

03

Compor objetos em árvores para representar hierarquias parte-todo e tratá-los uniformemente

AD BR CP DC FC FW PX

Motivação: pastas e arquivos no explorador



Sistema de arquivos

No explorador de arquivos, uma pasta pode conter arquivos e outras pastas, que por sua vez contêm mais arquivos e mais pastas, em qualquer profundidade.

Você clica em “Propriedades” e quer o tamanho. Para um arquivo, é o número de bytes; para uma pasta, é a soma de tudo que existe dentro dela.

Para o usuário, é a mesma pergunta: “qual o tamanho disto?”. O código deveria enxergar assim também.

> *Como escrever o cálculo do tamanho sem perguntar o tempo todo “isto é um arquivo ou uma pasta?”*

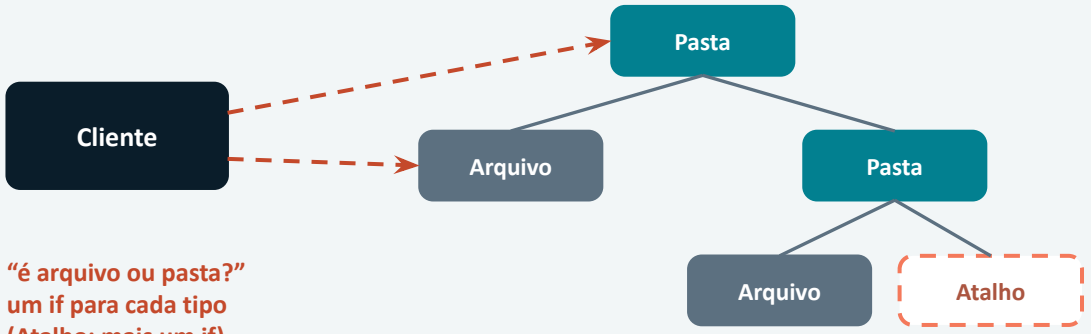
O problema: o cliente precisa saber quem é quem

```
long tamanho(Object item) {
    if (item instanceof Arquivo) {
        return ((Arquivo) item).bytes();
    }
    long total = 0;
    for (Object filho : ((Pasta) item).filhos()) {
        total += tamanho(filho);    // e se existir um Atalho? E um Link?
    }
    return total;
}
```

✕ Sem o padrão

- instanceof + cast para distinguir folha de contêiner
- Cada operação nova (contar, buscar, imprimir) repete a travessia
- Novo tipo de item (Atalho) obriga a alterar todos os clientes
- Arquivo e Pasta não compartilham nenhuma interface

O QUE ESTÁ ACONTECENDO



"é arquivo ou pasta?"
um if para cada tipo
(Atalho: mais um if)

Intenção: tratar parte e todo da mesma forma

> *"Compor objetos em estruturas de árvore para representar hierarquias parte-todo. O Composite permite que os clientes tratem objetos individuais e composições de objetos de maneira uniforme."*

GAMMA, HELM, JOHNSON, VLISSIDES (1994) — Intent (tradução livre)

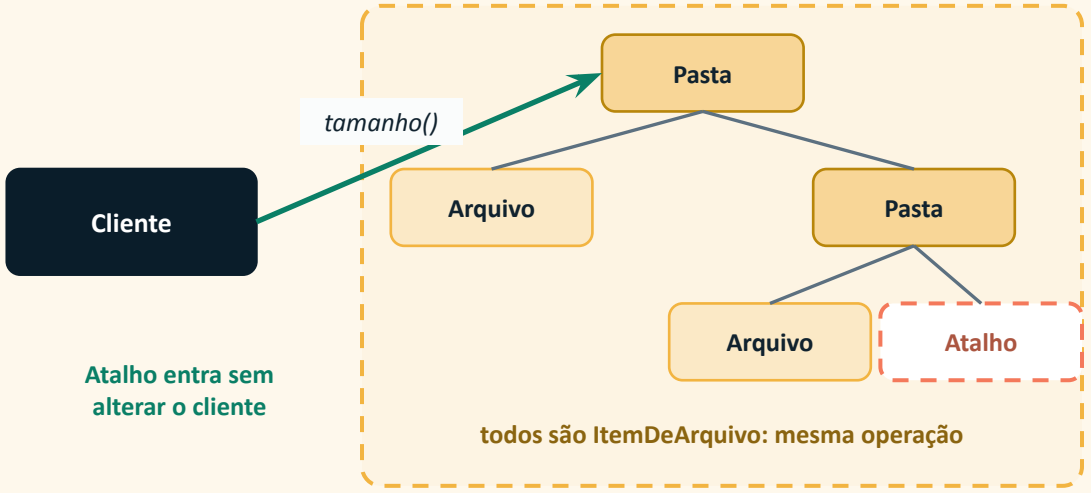
Folhas e compostos implementam a mesma interface (o Component). O composto guarda uma lista de Components e repassa a operação a cada filho.

O cliente chama uma única operação na raiz; a recursão percorre a árvore toda por polimorfismo.

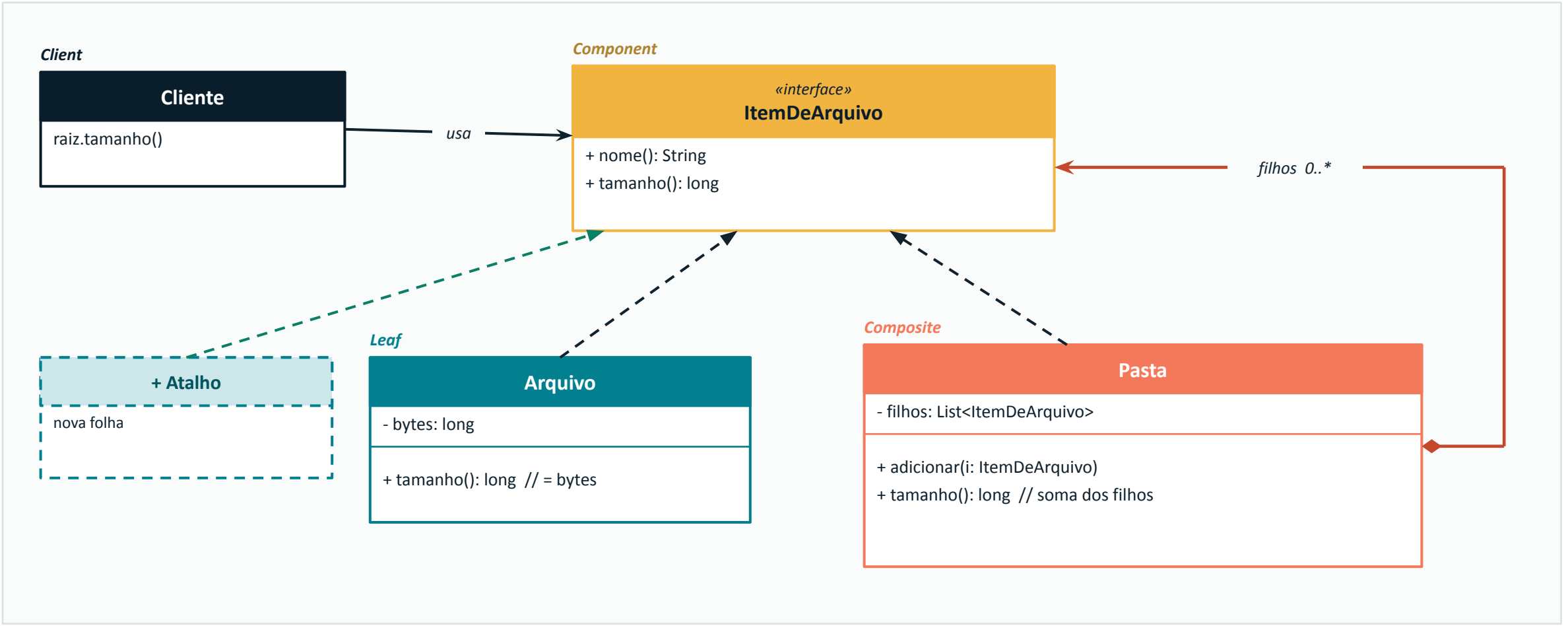
PAPÉIS DO PADRÃO

- **Component:** interface comum (ItemDeArquivo).
- **Leaf:** objeto sem filhos (Arquivo).
- **Composite:** objeto com filhos, que delega a eles (Pasta).

A SOLUÇÃO EM AÇÃO



Estrutura: Pasta contém ItemDeArquivo (inclusive outras Pastas)



A seta com losango é a recursão: Pasta contém 0.. ItemDeArquivo, e Pasta também é um ItemDeArquivo.*

Código: Pasta delega tamanho() a cada filho

ItemDeArquivo.java • Arquivo.java

```java
// Component — interface comum a folhas e compostos
public interface ItemDeArquivo {
    String nome();
    long tamanho();
}

// Leaf — não tem filhos
public class Arquivo implements ItemDeArquivo {
    private final String nome;
    private final long bytes;

    public Arquivo(String nome, long bytes) {
        this.nome = nome;
        this.bytes = bytes;
    }
    public String nome()  { return nome; }
    public long tamanho() { return bytes; }
}
```

Pasta.java • Cliente

```java
// Composite — contém outros itens (inclusive Pastas)
public class Pasta implements ItemDeArquivo {
    private final String nome;
    private final List<ItemDeArquivo> filhos = new
ArrayList<>();

    public Pasta(String nome) { this.nome = nome; }
    public void adicionar(ItemDeArquivo i) { filhos.add(i); }
    public String nome() { return nome; }
    public long tamanho() {
        long total = 0;
        for (ItemDeArquivo f : filhos) total += f.tamanho();
        return total;           // recursão polimórfica}}
Pasta docs = new Pasta("docs");
docs.adicionar(new Arquivo("a.pdf", 2_000));
docs.adicionar(new Arquivo("b.txt", 500));
docs.tamanho();   // 2500
```

raiz.tamanho() funciona igual para Arquivo e para Pasta, em qualquer profundidade. Um Atalho novo só precisa implementar ItemDeArquivo.

Quando usar (e quando evitar) o Composite

✓ Use quando

- Há uma hierarquia parte-todo: arquivos e pastas, menus, componentes de interface, organogramas, expressões
- Os clientes devem tratar objeto simples e composição de forma uniforme
- As operações são naturalmente recursivas: somar, buscar, desenhar, imprimir
- A estrutura é uma árvore, e a profundidade não é fixa

! Cuidado / evite quando

- Interface comum larga demais: a folha herda adicionar()/remover() sem sentido (transparência × segurança)
- Difícil restringir quais filhos um composto aceita; as checagens ficam em tempo de execução
- Árvores muito profundas ou grandes: custo da travessia recursiva
- Se a estrutura é uma lista plana, o padrão é exagero

PADRÕES RELACIONADOS

Decorator (estrutura recursiva parecida, mas com um único filho e foco em acrescentar responsabilidade) • Iterator e Visitor (percorrer e operar sobre a árvore) • Flyweight (compartilhar folhas) • Chain of Responsibility

PADRÃO ESTRUTURAL 4 DE 7

Decorator

04

Acrescentar responsabilidades a um objeto dinamicamente, como alternativa flexível à herança

AD BR CP DC FC FW PX

Motivação: montando o pedido na cafeteria



Cafeteria

Na cafeteria você pede um café e personaliza: com leite, com chocolate, com chantilly. Cada extra tem seu preço e a conta soma todos.

Os clientes combinam os extras como quiserem: só leite, leite e chocolate, os três juntos, chocolate em dobro…

O extra não muda o que o produto é: continua sendo uma bebida com descrição e preço. Ele apenas acrescenta algo por cima.

> *Como representar um café com qualquer combinação de extras sem criar uma classe para cada combinação possível?*

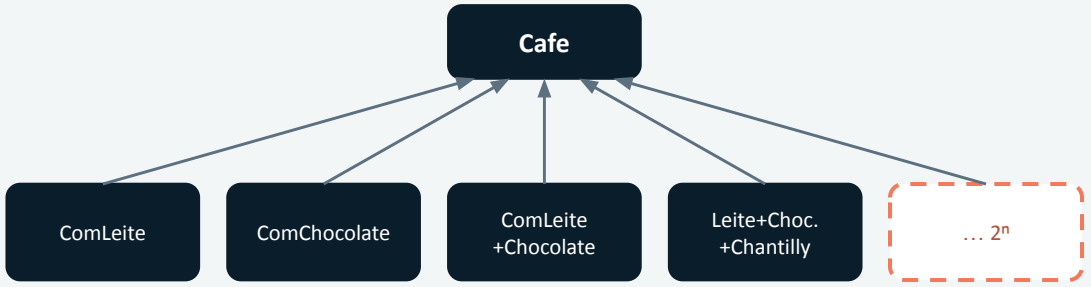
O problema: cada combinação vira uma subclasse

```
class Cafe { double preco() { return 5.00; } }
class CafeComLeite extends Cafe { double preco() { return 6.50; } }
class CafeComChocolate extends Cafe { double preco() { return 7.00; } }
class CafeComLeiteEChocolate extends Cafe { double preco() { return 8.50; } }
class CafeComLeiteChocolateEChantilly extends Cafe { double preco() { return 10.00; } }
// ... e CafeComChantilly, CafeComLeiteEChantilly, CafeComChocolateEChantilly ...
// n extras  →  2ⁿ combinações.  Mudar o preço do leite = editar várias classes.
```

✕ Sem o padrão

- n extras geram 2^n subclasses (3 extras = 8; 6 extras = 64)
- O preço do leite está repetido em várias classes
- A combinação é fixada em tempo de compilação: o cliente não monta o pedido
- Extra em dobro (chocolate duas vezes)? Outra classe

O QUE ESTÁ ACONTECENDO



cada caixa é uma classe: preço e descrição repetidos

Intenção: embrulhar o objeto para acrescentar comportamento

> *"Atribuir responsabilidades adicionais a um objeto dinamicamente. Os decorators fornecem uma alternativa flexível à herança para estender funcionalidades."*

GAMMA, HELM, JOHNSON, VLISSIDES (1994) — Intent (tradução livre)

O decorator tem a mesma interface do objeto que embrulha e guarda uma referência a ele. Cada chamada é repassada ao objeto de dentro, e o decorator acrescenta o seu trabalho antes ou depois.

Decorators se empilham: o resultado de um embrulho é o objeto que outro embrulho recebe.

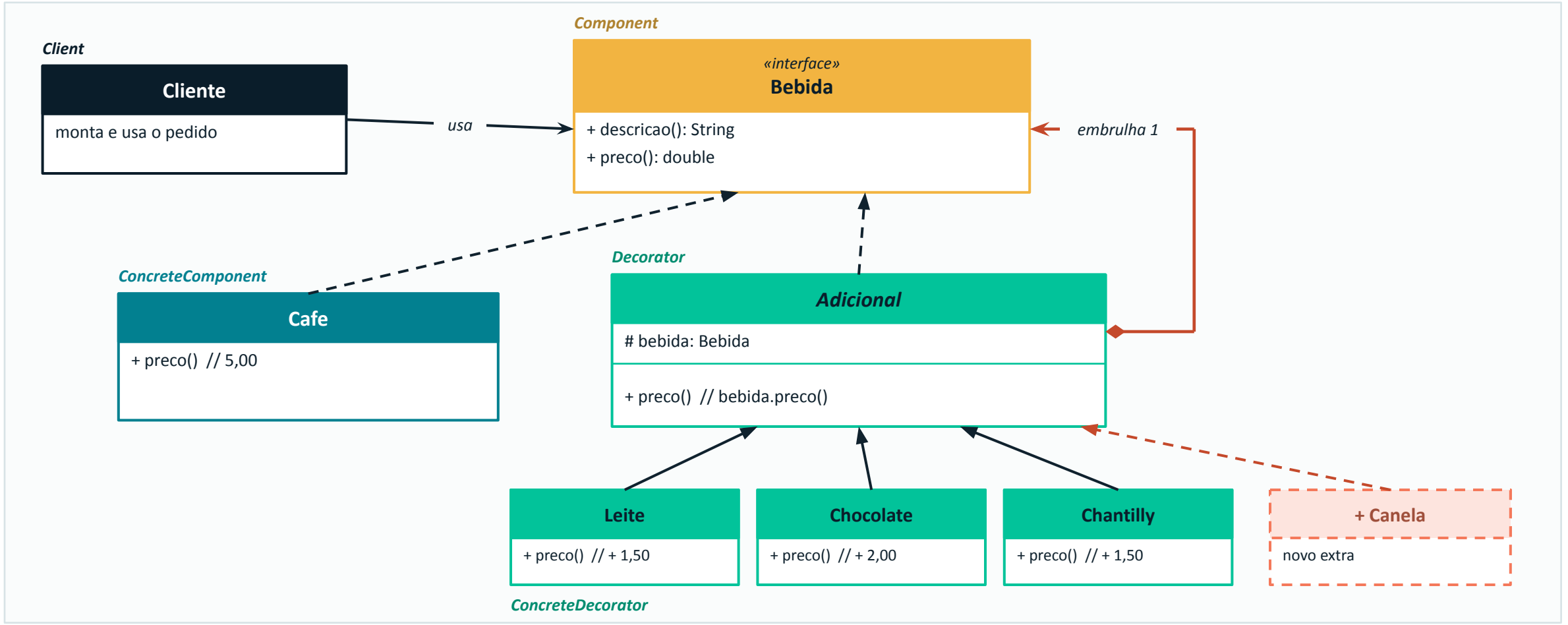
POR QUE NÃO HERANÇA?

- **Herança é estática:** decidida na compilação, vale para toda a classe.
- **Decorator é dinâmico:** montado em execução, objeto a objeto.

A SOLUÇÃO EM AÇÃO



Estrutura: Adicional é uma Bebida que contém outra Bebida



Extra novo = 1 classe nova que estende Adicional. O Cafe e os outros extras permanecem intactos.

Código: cada Adicional embrulha uma Bebida

Bebida.java • Cafe.java • Adicional.java

```java
// Component
public interface Bebida {
    String descricao();
    double preco();
}
// ConcreteComponent — o objeto "base"
public class Cafe implements Bebida {
    public String descricao() { return "Café"; }
    public double preco()     { return 5.00; }
}
// Decorator — mesma interface + referência a outra Bebida
public abstract class Adicional implements Bebida {
    protected final Bebida bebida;
    protected Adicional(Bebida bebida) { this.bebida = bebida;}

    public String descricao() { return bebida.descricao(); }
    public double preco()     { return bebida.preco(); }
```

Leite.java • Cliente

```java
// ConcreteDecorator — acrescenta uma responsabilidade
public class Leite extends Adicional {
    public Leite(Bebida b) { super(b); }

    @Override public String descricao() {
        return bebida.descricao() + " + leite";
    }
    @Override public double preco() {
        return bebida.preco() + 1.50;
    }
}
Bebida pedido = new Chantilly(
        new Chocolate(new Leite(new Cafe())));

pedido.descricao(); // Café + leite + chocolate + chantilly
pedido.preco();     // 10.00
```

A ordem dos embrulhos define a pilha de chamadas. Um extra novo (Canela) é uma classe nova; nenhuma classe existente muda.

Quando usar (e quando evitar) o Decorator

✓ Use quando

- Acrescentar responsabilidades a objetos individuais, em tempo de execução, sem afetar os outros
- Evitar a explosão de subclasses para combinações de comportamentos
- A extensão por herança é impraticável (classe final, combinações demais)
- Empilhar comportamentos transversais: compressão, criptografia, log, cache

! Cuidado / evite quando

- Muitos decorators pequenos: pilhas longas, difíceis de depurar (stack traces enormes)
- A ordem importa: criptografar → comprimir não é o mesmo que comprimir → criptografar
- Identidade: o decorator não é o objeto original (equals, instanceof)
- Remover um decorator do meio da pilha é difícil

PADRÕES RELACIONADOS

Composite (estrutura recursiva parecida, mas o Decorator tem um só filho e acrescenta responsabilidade) • Strategy (troca o “miolo”; o Decorator troca a “pele”) • Proxy (mesma estrutura, mas controla o acesso) • Adapter (muda a interface; o Decorator a mantém)

PADRÃO ESTRUTURAL 5 DE 7

Facade

05

Oferecer uma interface única e simples para um conjunto de interfaces de um subsistema

AD | BR | CP | DC | **FC** | FW | PX

Motivação: assistir a um filme no home theater



Home theater

Para assistir a um filme em casa você precisa: diminuir as luzes, baixar a tela, ligar o projetor e escolher a entrada, ligar o amplificador em modo surround, ajustar o volume e abrir o aplicativo de streaming.

São nove passos em cinco aparelhos, sempre na mesma ordem. Ao terminar, outros cinco passos para desfazer tudo.

Você quer apenas "assistir a um filme". Por isso existe o botão único no controle universal, que faz a sequência por você.

> *Em software, quem usa o home theater também só quer "assistir". O que acontece se cada tela do aplicativo controlar os cinco aparelhos?*

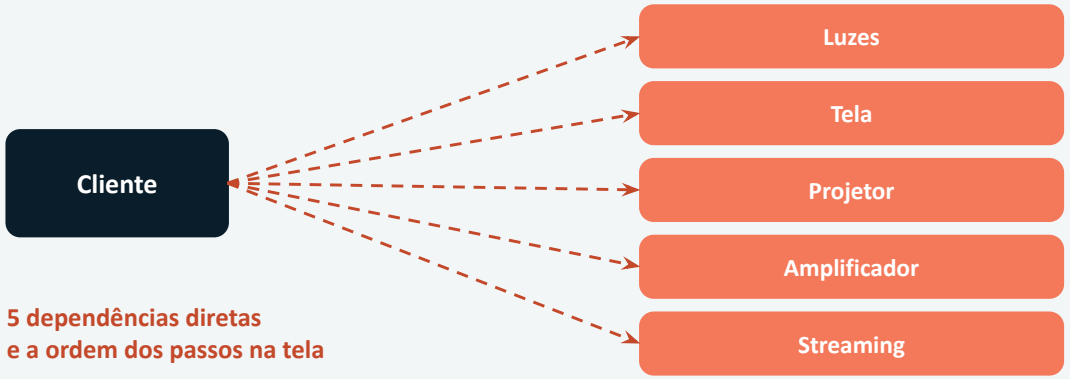
O problema: o cliente controla todos os aparelhos

```
// Tela do aplicativo: conhece os cinco aparelhos e a ordem correta
luzes.diminuir(10);
tela.baixar();
projetor.ligar();            projetor.definirEntrada("HDMI1");
amplificador.ligar();        amplificador.definirModo("SURROUND");
amplificador.definirVolume(15);
streaming.abrir();           streaming.reproduzir("Matrix");
// ... e o mesmo bloco copiado na tela de agendamento, no modo noturno, no app do tablet
```

✕ Sem o padrão

- Cliente acoplado a 5 classes do subsistema
- A ordem dos passos é conhecimento espalhado e duplicado em cada tela
- Trocar um aparelho quebra todos os clientes
- Difícil de testar e de reutilizar

O QUE ESTÁ ACONTECENDO



Intenção: uma porta de entrada simples para o subsistema

> *"Fornecer uma interface unificada para um conjunto de interfaces em um subsistema. O Facade define uma interface de nível mais alto que torna o subsistema mais fácil de usar."*

GAMMA, HELM, JOHNSON, VLISSIDES (1994) — Intent (tradução livre)

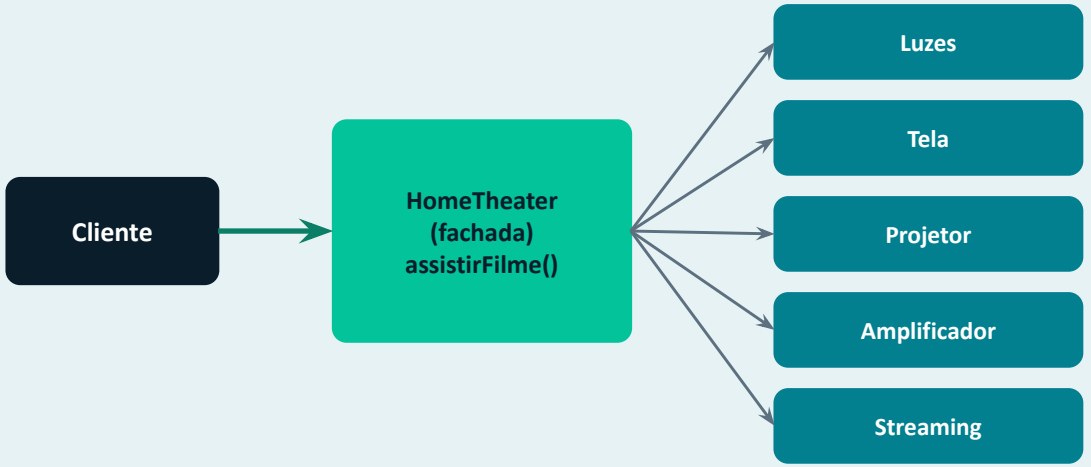
A fachada conhece as classes do subsistema, a ordem correta das chamadas e as configurações. Ela expõe poucas operações de alto nível, como assistirFilme().

O subsistema não sabe que a fachada existe: nada nele é alterado.

O QUE O FACADE NÃO FAZ

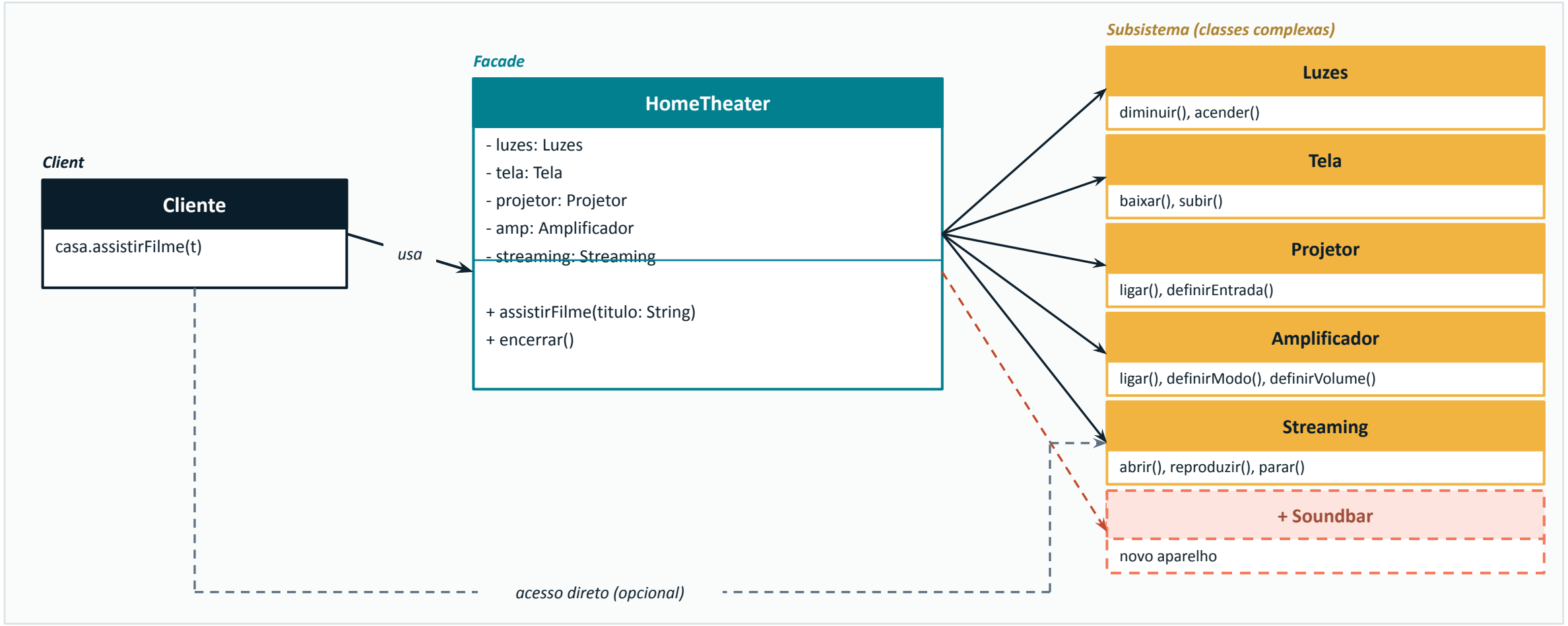
- **Não proíbe o acesso direto:** quem precisar de controle fino ainda usa as classes internas.
- **Não deve ter regras de negócio:** apenas coordena e delega.

A SOLUÇÃO EM AÇÃO



1 chamada para o cliente; as 9 do subsistema ficam dentro da fachada

Estrutura: HomeTheater coordena as classes do subsistema



Aparelho novo (Soundbar) = alteração só na fachada. O acesso direto ao subsistema continua possível (seta tracejada).

Código: HomeTheater esconde a sequência de chamadas

HomeTheater.java (Facade)

```java
// Facade — interface simples para o subsistema
public class HomeTheater {
    private final Luzes luzes = new Luzes();
    private final Tela tela = new Tela();
    private final Projetor projetor = new Projetor();
    private final Amplificador amp = new Amplificador();
    private final Streaming streaming = new Streaming();

    public void assistirFilme(String titulo) {
        luzes.diminuir(10);
        tela.baixar();
        projetor.ligar();
        projetor.definirEntrada("HDMI1");
        amp.ligar();
        amp.definirModo("SURROUND");
        amp.definirVolume(15);
        streaming.abrir();
        streaming.reproduzir(titulo);
    }
    public void encerrar() {
        streaming.parar();
        amp.desligar();
        projetor.desligar();
        tela.subir();
        luzes.acender();
    }
}

// Client — conhece apenas a fachada
HomeTheater casa = new HomeTheater();
casa.assistirFilme("Matrix");   // 1 chamada em vez de 9
casa.encerrar();

// Subsistema (assinaturas): Luzes.diminuir()/acender(),
// Tela.baixar()/subir(), Projetor.ligar()/desligar()/
// definirEntrada(), Amplificador.ligar()/desligar()/
// definirModo()/definirVolume(), Streaming.abrir()/
```

HomeTheater.java (cont.) • Cliente

A fachada pode criar ou receber (injeção) o subsistema. Quem precisar de controle fino continua usando as classes internas diretamente.

Quando usar (e quando evitar) o Facade

✓ Use quando

- Oferecer uma interface simples para um subsistema complexo
- Reduzir o acoplamento entre os clientes e muitas classes do subsistema
- Estruturar o sistema em camadas: uma fachada como ponto de entrada de cada camada ou módulo
- Isolar o código da aplicação de bibliotecas de terceiros complicadas

! Cuidado / evite quando

- A fachada vira um “objeto-deus”: acumula regras de negócio em vez de apenas delegar
- Esconder demais: clientes avançados ficam sem acesso ao que precisam
- Vazar tipos internos do subsistema na assinatura da fachada mantém o acoplamento
- Subsistema simples não precisa de fachada

PADRÕES RELACIONADOS

Adapter (adapta uma interface existente; o Facade cria uma interface nova e mais simples) • Mediator (coordena colegas em comunicação bidirecional) • Abstract Factory (alternativa para esconder a criação do subsistema) • Singleton (fachadas costumam ter instância única)

PADRÃO ESTRUTURAL 6 DE 7

Flyweight

Usar compartilhamento para suportar eficientemente um grande número de objetos de granularidade fina

06

AD | BR | CP | DC | FC | **FW** | PX

Motivação: a floresta de um jogo de mundo aberto



Floresta num jogo

Um jogo de mundo aberto desenha uma floresta com 100.000 árvores. Existem poucos tipos: pinheiro, carvalho, palmeira.

Todos os pinheiros usam a mesma malha 3D e a mesma textura (dados pesados, cerca de 2 MB, valores ilustrativos). O que muda de árvore para árvore é só a posição (x, y), que é pequena.

O que é caro é idêntico em milhares de árvores. Guardar uma cópia por árvore desperdiça memória e tempo de carga.

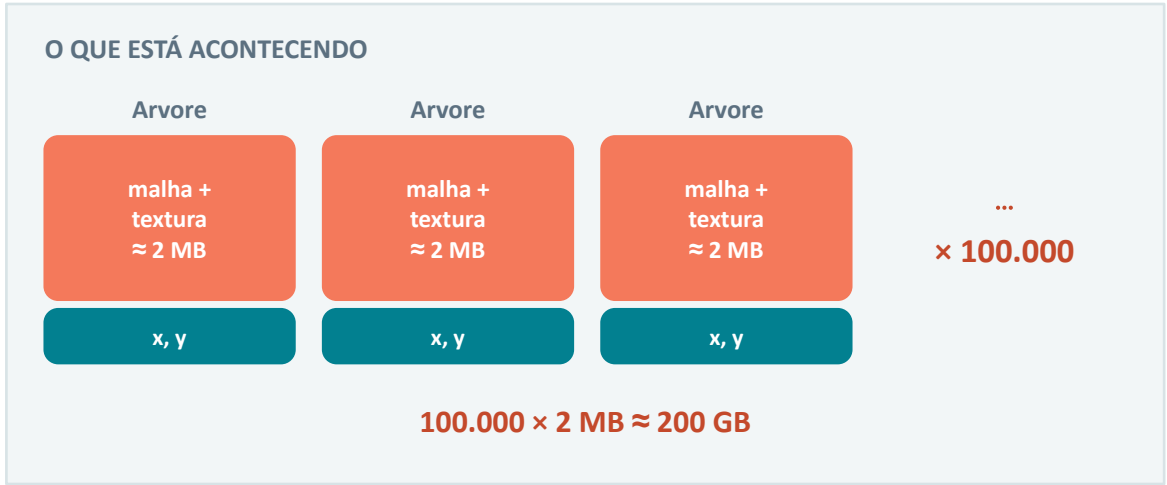
> *O que acontece se cada uma das 100.000 árvores carregar a sua própria malha e a sua própria textura?*

O problema: cada árvore carrega a sua própria cópia

```
class Arvore {
    String nome;  Textura textura;  Malha malha;     // ~2 MB (igual para todo pinheiro!)
    int x, y;                                         // ~8 bytes (único de cada árvore)
    Arvore(String nome, int x, int y) {
        this.nome = nome;  this.x = x;  this.y = y;
        this.textura = Textura.carregar(nome + ".png");  // lê do disco, toda vez
        this.malha   = Malha.carregar(nome + ".obj");    // lê do disco, toda vez
    }
}   // for (i < 100_000) arvores.add(new Arvore("pinheiro", rnd(), rnd()));
```

✕ Sem o padrão

- 100.000 árvores × 2 MB ≈ 200 GB de memória (valores ilustrativos)
- Dados idênticos duplicados em cada instância
- Leitura repetida do disco: carga lentíssima
- Coletor de lixo e cache da CPU sofrem com tantos objetos grandes



Intenção: separar o que é compartilhado do que é único

> *"Usar compartilhamento para suportar eficientemente um grande número de objetos de granularidade fina."*

GAMMA, HELM, JOHNSON, VLISSIDES (1994) — Intent (tradução livre)

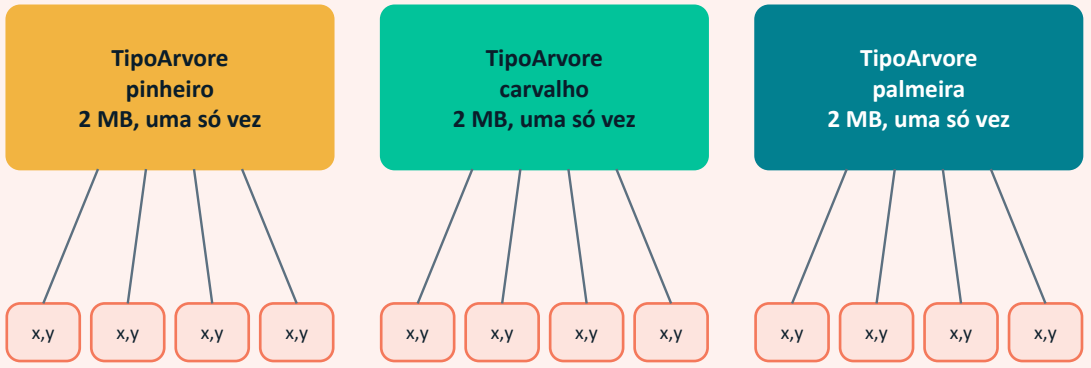
Dividimos o estado do objeto em duas partes. O estado intrínseco é igual para muitos objetos e vai para um flyweight compartilhado. O estado extrínseco muda de objeto para objeto e fica fora, no contexto.

Uma fábrica garante o compartilhamento: só cria um novo flyweight quando ainda não existe.

REGRA DE OURO

- **O flyweight deve ser imutável:** vários objetos o usam ao mesmo tempo.

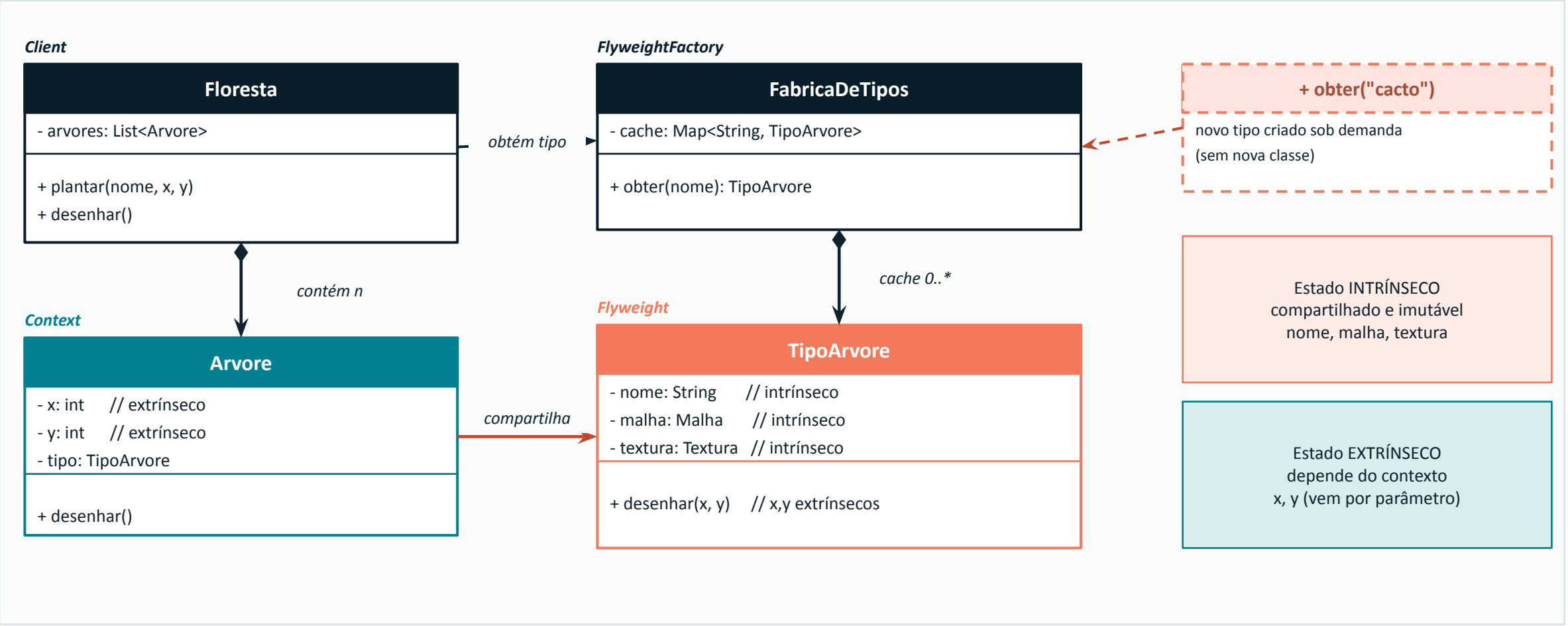
A SOLUÇÃO EM AÇÃO



⋯ 100.000 árvores guardam só (x, y) e uma referência

3 × 2 MB + 100.000 × 8 B ≈ 7 MB

Estrutura: Arvore guarda o contexto e referencia um TipoArvore



Cada árvore guarda só (x, y) + uma referência. A fábrica garante que existe um único TipoArvore por nome.

Código: flyweight, fábrica e contexto

Estado intrínseco (compartilhado) no TipoArvore; estado extrínseco (x, y) na Arvore.

TipoArvore.java (Flyweight)

```java
// Flyweight — estado INTRÍNSECO (imutável, compartilhado)
public final class TipoArvore {
    private final String nome;
    private final Malha malha;
    private final Textura textura;

    public TipoArvore(String nome) {
        this.nome = nome;
        this.malha = Malha.carregar(nome + ".obj");
        this.textura = Textura.carregar(nome + ".png");
    }

    // x e y chegam por parâmetro: estado EXTRÍNSECO
    public void desenhar(int x, int y) {
        Canvas.desenhar(malha, textura, x, y);
    }
}
```

FabricaDeTipos.java • Arvore.java

```java
// FlyweightFactory — garante o compartilhamento
public class FabricaDeTipos {
    private static final Map<String, TipoArvore> cache =
            new HashMap<>();

    public static TipoArvore obter(String nome) {
        return cache.computeIfAbsent(nome, TipoArvore::new);
    }
}

// Context — estado extrínseco + referência ao flyweight
public class Arvore {
    private final int x, y;
    private final TipoArvore tipo;       // compartilhado

    public Arvore(String nome, int x, int y) {
        this.tipo = FabricaDeTipos.obter(nome);
        this.x = x;  this.y = y;
    }
    public void desenhar() { tipo.desenhar(x, y); }
}
```

Código: o cliente planta árvores de tipos diferentes

Floresta.java • Main.java (Client)

```java
// Client — guarda as árvores e pede para desenhá-las
public class Floresta {
    private final List<Arvore> arvores = new ArrayList<>();
    public void plantar(String nome, int x, int y) {
        arvores.add(new Arvore(nome, x, y));
    }
    public void desenhar() { arvores.forEach(Arvore::desenhar); }
}

public class Main {
    public static void main(String[] args) {
        Floresta floresta = new Floresta();
        floresta.plantar("pinheiro", 10, 20);   // cache vazio  → cria TipoArvore "pinheiro"
        floresta.plantar("carvalho",  35,  8);  //              → cria TipoArvore "carvalho"
        floresta.plantar("pinheiro", 60, 44);   //              → REUTILIZA "pinheiro"
        floresta.plantar("palmeira",   5, 90);  //              → cria TipoArvore "palmeira"
        floresta.plantar("pinheiro", 72, 15);   //              → REUTILIZA "pinheiro"
        floresta.desenhar();
```

5 árvores, apenas 3 TipoArvore na memória. Com 100.000 árvores, continuam sendo 3 tipos: o cliente só passa o nome e a posição.

Quando usar (e quando evitar) o Flyweight

✓ Use quando

- A aplicação cria um número enorme de objetos parecidos e a memória é o gargalo
- Boa parte do estado do objeto pode ser tornada extrínseca
- Muitos grupos de objetos podem ser trocados por poucos objetos compartilhados
- A identidade do objeto não importa para a aplicação

! Cuidado / evite quando

- Se a memória não é problema, a complexidade extra não compensa: meça antes de aplicar
- O estado extrínseco passado por parâmetro custa CPU e polui as assinaturas
- Flyweights precisam ser imutáveis; só assim o compartilhamento é seguro em concorrência
- Depuração mais difícil: muitos objetos apontam para a mesma instância

PADRÕES RELACIONADOS

Composite (as folhas costumam ser flyweights) • State e Strategy (objetos de estado e de estratégia costumam ser flyweights) • Singleton (a fábrica costuma ser única) • Factory Method (cria os flyweights). No JDK: Integer.valueOf (cache de −128 a 127) e o pool de Strings.

PADRÃO ESTRUTURAL 7 DE 7

Proxy

07

Fornecer um substituto ou representante de outro objeto para controlar o acesso a ele

AD BR CP DC FC FW PX

Motivação: a galeria de fotos em alta resolução



Galeria de fotos

Um álbum digital mostra 500 fotos em alta resolução, cada uma com cerca de 20 MB (valores ilustrativos), guardadas no disco ou na nuvem.

Ao abrir o álbum, o usuário vê uma grade com espaços reservados para as fotos. Só quando ele toca em uma foto é que ela realmente precisa ser lida e desenhada.

A grade só precisa de “algo que represente a foto”: o conteúdo pesado pode ser carregado depois, e só se for pedido.

> *O que acontece se o álbum criar as 500 fotos de verdade assim que for aberto?*

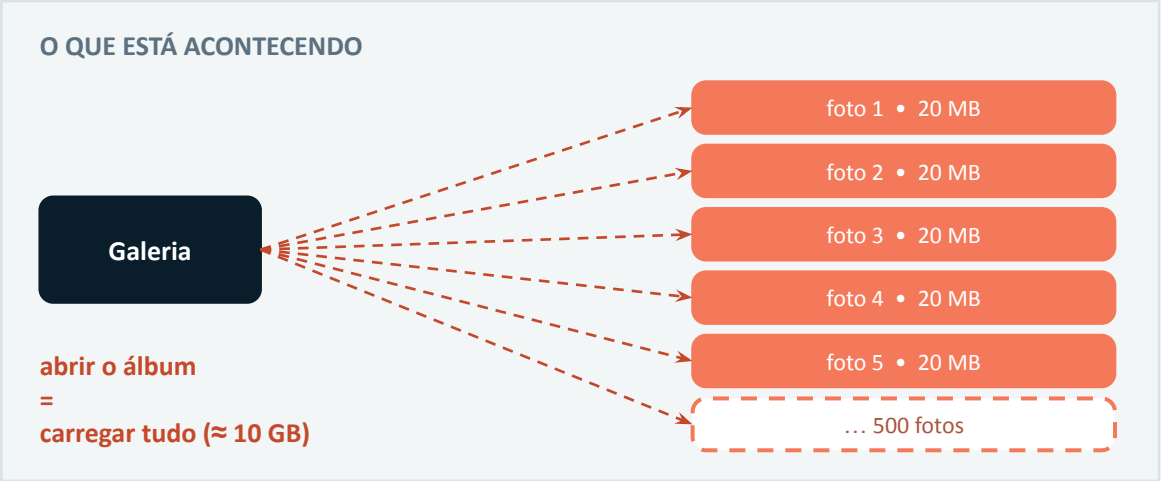
O problema: carregar tudo antes de precisar

```
class Galeria {
    private final List<Imagem> fotos = new ArrayList<>();

    Galeria(List<String> arquivos) {
        for (String arq : arquivos)
            fotos.add(new ImagemReal(arq));    // lê ~20 MB do disco/rede AQUI, no construtor
    }
    void abrir(int i) { fotos.get(i).exibir(); }    // o usuário vê 1 foto por vez
}
```

✕ Sem o padrão

- Abrir o álbum de 500 fotos carrega ≈ 10 GB (valores ilustrativos)
- Tela congelada durante a carga
- Fotos que ninguém abre ocupam memória
- Se o cliente tentar “consertar” com if (carregada), a lógica de carga tardia se espalha pelo código



Intenção: um substituto que controla o acesso

> *"Fornecer um substituto ou ponto de acesso a outro objeto para controlar o acesso a ele."*

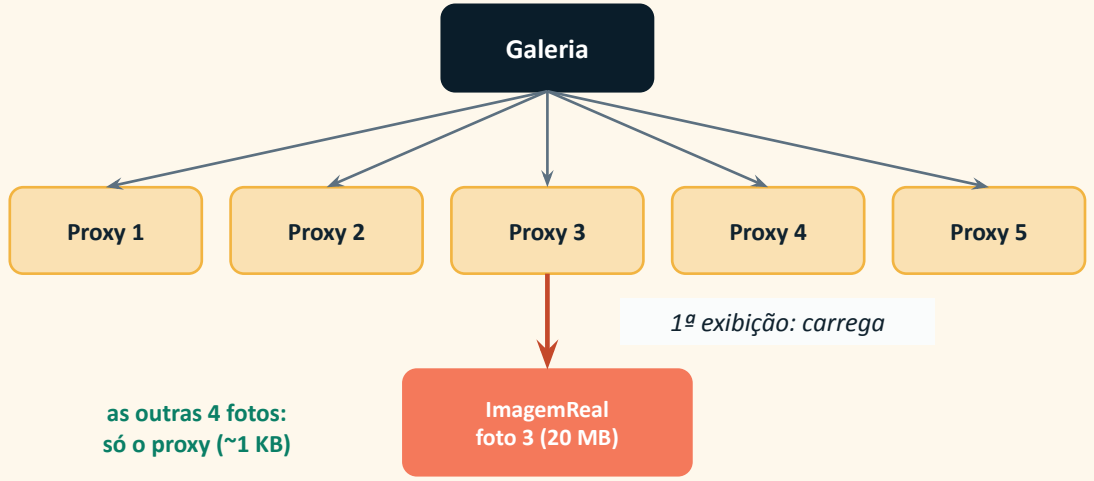
GAMMA, HELM, JOHNSON, VLISSIDES (1994) — Intent (tradução livre)

O proxy implementa a mesma interface do objeto real e fica no lugar dele. O cliente não percebe a troca: o proxy decide quando, como e se repassa a chamada ao objeto real.

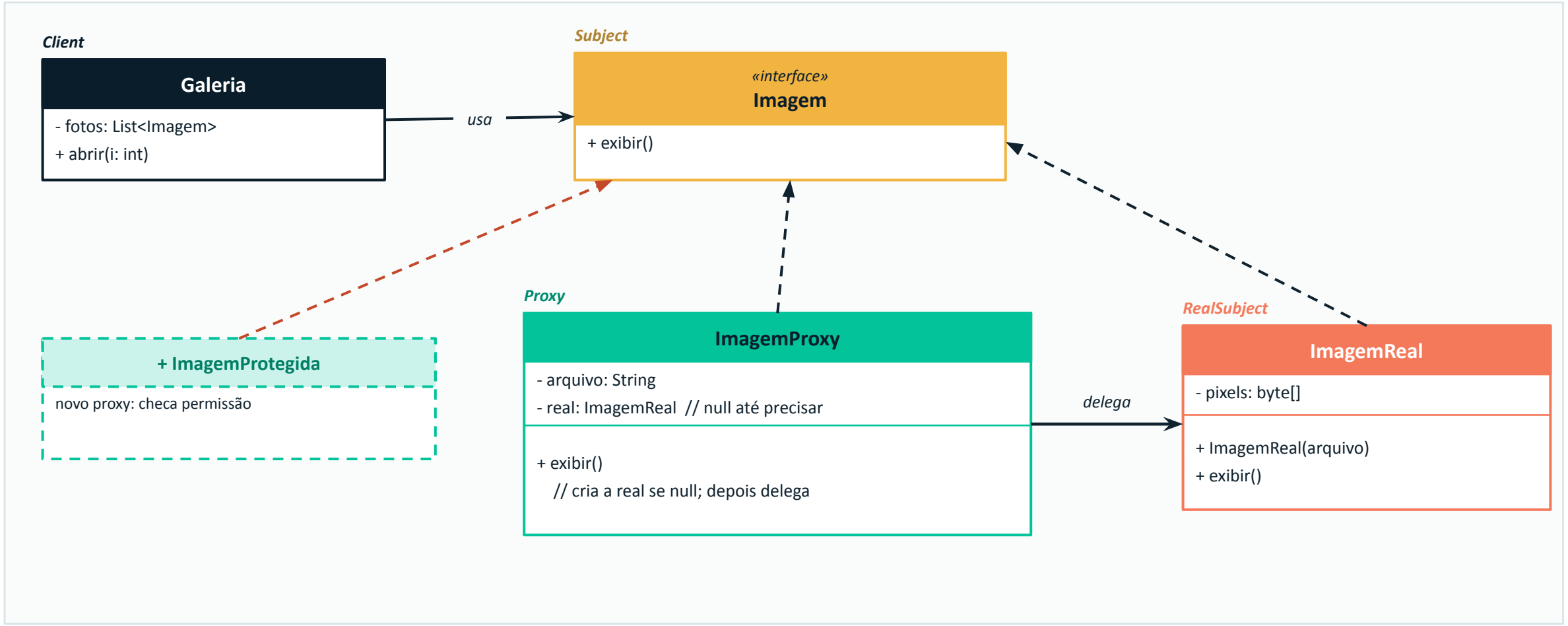
TIPOS DE PROXY

- **Virtual:** cria o objeto caro sob demanda (lazy loading).
- **De proteção:** verifica permissões antes de delegar.
- **Remoto:** representa localmente um objeto em outro processo ou servidor.
- **De cache ou log:** guarda resultados ou registra as chamadas.

A SOLUÇÃO EM AÇÃO



Estrutura: ImagemProxy e ImagemReal implementam Imagem



O cliente depende só de Imagem. Proxy de proteção, de cache ou de log são novas implementações da mesma interface.

Código: ImagemProxy cria a ImagemReal só na primeira exibição

Imagem.java • ImagemReal.java

```java
// Subject — contrato comum
public interface Imagem {
    void exibir();
}
// RealSubject — objeto caro de criar
public class ImagemReal implements Imagem {
    private final String arquivo;
    private final byte[] pixels;

    public ImagemReal(String arquivo) {
        this.arquivo = arquivo;
        this.pixels = Disco.ler(arquivo);   // ~20 MB, lento
    }
    @Override
    public void exibir() {
        Tela.desenhar(pixels);
    }
}
```

ImagemProxy.java • Cliente

```java
// Proxy — mesma interface, controla o acesso
public class ImagemProxy implements Imagem {
    private final String arquivo;
    private ImagemReal real;          // criada sob demanda

    public ImagemProxy(String a) { this.arquivo = a; }
    @Override
    public void exibir() {
        if (real == null) {                     // lazy loading
            real = new ImagemReal(arquivo);
        }
        real.exibir();}}
// Client — não percebe a diferença
List<Imagem> fotos = new ArrayList<>();
for (String arq : arquivos) fotos.add(new ImagemProxy(arq));
fotos.get(42).exibir();   // só agora a foto 42 é lida
```

A Galeria passa a criar 500 proxies baratos em vez de 500 imagens pesadas. Proxy de proteção ou de cache segue o mesmo molde.

Quando usar (e quando evitar) o Proxy

✓ Use quando

- Inicialização preguiçosa de objetos caros (proxy virtual)
- Controlar o acesso antes de delegar: permissões, limites (proxy de proteção)
- Representante local de um objeto remoto (RMI, stubs de gRPC)
- Cache, log, contagem de acessos e transações, como nos proxies do Spring AOP e do Hibernate

! Cuidado / evite quando

- Latência e indireção extras em chamadas que já são baratas
- O proxy pode ficar fora de sincronia com o objeto real (por exemplo, com cache desatualizado)
- Proxies dinâmicos (java.lang.reflect.Proxy) dificultam a depuração e a leitura de stack traces
- Se o objetivo é só acrescentar comportamento empilhável, o Decorator é mais indicado

PADRÕES RELACIONADOS

Decorator (mesma estrutura; o Decorator acrescenta comportamento, o Proxy controla o acesso e costuma gerenciar o ciclo de vida do objeto real) • Adapter (muda a interface; o Proxy a mantém) • Facade • Flyweight

SÍNTESE

Onde os estruturais já aparecem no JDK e em frameworks

Classificações usuais da literatura; muitas APIs misturam mais de um padrão.

Padrão	Onde você já viu	Como reconhecer
Adapter	InputStreamReader (InputStream → Reader); Arrays.asList (array → List)	Classe que traduz uma API para outra
Bridge	JDBC: Connection e Statement (abstração) × o Driver de cada banco (implementação)	Interface estável + implementações plugáveis
Composite	java.awt.Container e Component (AWT e Swing); Node e Parent (JavaFX)	Contêiner que também é um componente
Decorator	BufferedInputStream, GZIPOutputStream; Collections.unmodifiableList	Construtor recebe o mesmo tipo que a classe implementa
Facade	SLF4J (fachada para Logback, Log4j e outros)	Uma API simples na frente de várias bibliotecas
Flyweight	Integer.valueOf (cache de −128 a 127); String.intern (pool de Strings)	Cache de instâncias imutáveis
Proxy	java.lang.reflect.Proxy; Spring AOP; proxies de carga tardia do Hibernate	Mesma interface, mas adia ou controla a chamada

SÍNTESE

Sinais de alerta: quando o código pede um padrão estrutural

Padrão	Pergunta de projeto	Sinal de alerta no código
Adapter	Como usar uma API cuja interface não combina com a minha?	Cliente cheio de conversões de unidade, formato e códigos de uma API externa
Bridge	Como evitar combinações de dois eixos de variação?	Nomes como AlertaEmail, AlertaSms, LembreteEmail…
Composite	Como tratar parte e todo da mesma forma?	instanceof Arquivo / instanceof Pasta, com laços recursivos nos clientes
Decorator	Como somar comportamentos sem uma subclasse por combinação?	Subclasses com nomes compostos: CafeComLeiteEChocolate
Facade	Como esconder um subsistema complicado?	A mesma sequência de chamadas a 4 ou mais classes repetida em vários clientes
Flyweight	Como reduzir a memória de milhares de objetos parecidos?	Muitos objetos com campos idênticos; OutOfMemoryError; carga repetida de recursos
Proxy	Como adiar ou controlar o acesso a um objeto?	if (carregado) ou if (temPermissao) repetido antes de cada uso do objeto

SÍNTESE

Quem embrulha quem? Distinguindo os padrões parecidos

A estrutura (um objeto que guarda outro) é parecida. A intenção é o que diferencia.

Padrão	O que faz com o objeto embrulhado	Interface que expõe	Pergunta que o distingue
Adapter	Traduz as chamadas	Diferente da do objeto embrulhado	As interfaces são incompatíveis?
Decorator	Acrescenta responsabilidade	A mesma	Quero somar comportamento?
Proxy	Controla o acesso	A mesma	Quero decidir se e quando chamar o objeto real?
Facade	Simplifica e coordena vários objetos	Nova e mais simples	Quero esconder um subsistema?
Composite	Agrupa vários filhos	A mesma da folha	Quero tratar parte e todo igualmente?

SÍNTESE

Como os padrões se combinam

Entre os estruturais

- **Composite + Decorator:** mesma estrutura recursiva; um agrega muitos filhos, o outro embrulha um só.
- **Composite + Flyweight:** folhas compartilhadas reduzem a memória de árvores grandes.
- **Facade + Adapter:** a fachada pode usar adapters para falar com o subsistema.
- **Adapter × Bridge:** o Adapter conserta depois; o Bridge é planejado antes.

Com os criacionais

- **Abstract Factory:** cria famílias de implementações para um Bridge.
- **Builder:** monta árvores Composite passo a passo.
- **Factory Method:** fábricas de Flyweight e de Proxy.
- **Singleton:** a Facade e a fábrica de Flyweight costumam ser únicas.

Atividade em sala e referências

ATIVIDADE (SUGESTÃO: EM DUPLAS)

1

Diagnóstico

Escolha um dos projetos de API da disciplina (por exemplo, Open-Meteo, NASA NeoWs ou MusicBrainz) e identifique dois sinais de alerta da tabela.

2

Refatoração

Aplique um padrão estrutural e esboce o UML e o código. Ideias: Adapter para o cliente HTTP, Facade para um fluxo de várias chamadas, Proxy para cache de respostas.

3

Discussão

Quais são os trade-offs? Existe uma solução mais simples? Qual padrão parecido você descartou e por quê?

REFERÊNCIAS

- GAMMA, E.; HELM, R.; JOHNSON, R.; VLISSIDES, J. Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley, 1994.
- FREEMAN, E.; ROBSON, E. Head First Design Patterns. 2. ed. O'Reilly, 2020.
- BLOCH, J. Effective Java. 3. ed. Addison-Wesley, 2018. (Item 18: dê preferência à composição em vez da herança)
- Refactoring.Guru. Design Patterns. refactoring.guru/design-patterns